

ETEC VASCO ANTÔNIO VENCHIARUTTI

IGOR DANIEL RODRIGUES EUSTACHIO – 2C1

Criptografia em PHP

JUNDIAÍ

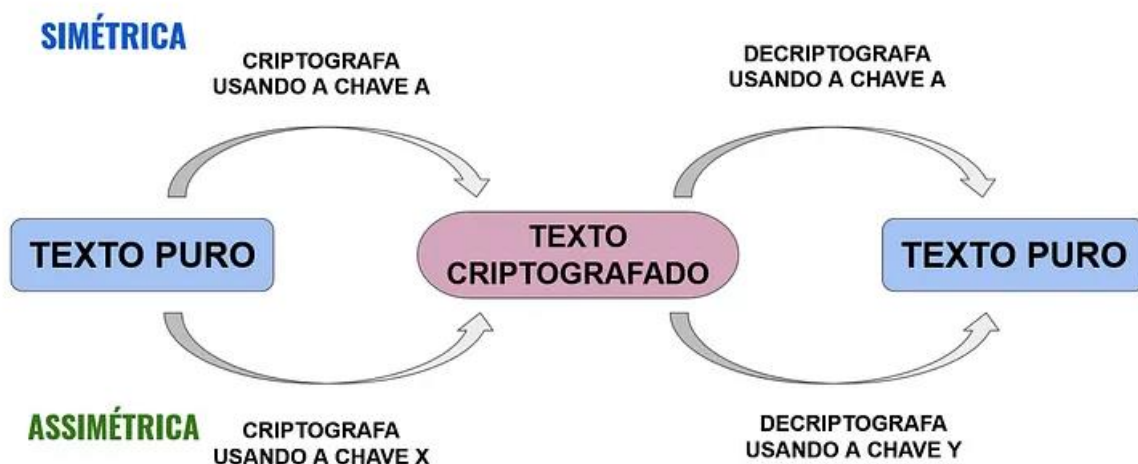
INTRODUÇÃO

A segurança e a criptografia de dados tornaram-se requisitos essenciais na construção de sistemas confiáveis. Embora técnicas para quebrar sistemas criptográficos sejam exploradas e evoluam constantemente, é imprescindível utilizar o maior nível de proteção ao nosso alcance para tornar as aplicações o mais seguras possível, aliando essas práticas a defesas complementares que impedem ataques comuns, como o *SQL Injection*.

O QUE É CRIPTOGRAFIA

A criptografia é a prática e o estudo de técnicas voltadas para assegurar a comunicação na presença de terceiros, por meio da construção e da análise de protocolos que impeçam agentes não autorizados de lerem mensagens privadas. O objetivo central desse processo consiste em transformar uma informação originalmente compreensível (chamada de texto claro) em um texto cifrado, tornando-a inteligível apenas para quem possui autorização e as chaves corretas para a reversão do processo. A criptografia se divide em dois tipos:

- Simétrica: usa a mesma chave para criptografar e descriptografar
- Assimétrica: usa uma chave para criptografar e outra diferente para descriptografar.



CRIPTOGRAFIA EM PHP

No PHP moderno, a segurança criptográfica é dividida em extensões nativas especializadas, categorizadas de acordo com o objetivo técnico da aplicação: encriptação reversível, hashing irreversível para senhas, verificação de integridade de dados e geração de aleatoriedade segura.

Encriptação Simétrica e Assimétrica (OpenSSL e Libsodium)

Para operações consideradas reversíveis, o PHP oferece duas ferramentas principais:

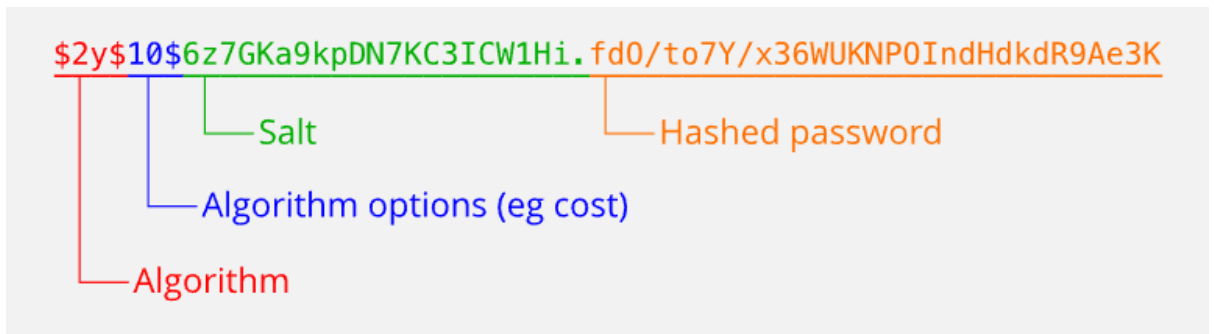
- OpenSSL: É o padrão tradicional da linguagem. Para criptografia simétrica, é usado `openssl_encrypt()` e `openssl_decrypt()`. A extensão também lida com criptografia assimétrica e assinaturas digitais por meio de `openssl_sign()` e `openssl_verify()`.
- Libsodium: Nativa no PHP, é a opção mais moderna e recomendada por ser safe-by-default. Ela facilita a configuração e oferece alto desempenho para criptografia simétrica (`sodium_crypto_secretbox()`) e assimétrica de chaves públicas (`sodium_crypto_box()`).



Armazenamento Seguro de Senhas (Password Hashing API)

Senhas de usuários nunca devem ser criptografadas de forma reversível e nem usando algoritmos rápidos, por isso o PHP possui ferramentas nativas exclusivas para senhas, que são lentas de propósito para impedir ataques:

- `password_hash()`: Transforma a senha em um hash (código irreversível) e cria automaticamente o salt (código aleatório de proteção).
- `password_verify()`: Compara de forma segura a senha digitada pelo usuário com o hash salvo no banco de dados.
- `password_needs_rehash()`: Verifica se o hash de uma senha antiga precisa ser atualizado caso o sistema mude o algoritmo ou aumente o nível de segurança.



Integridade e Autenticação de Mensagens (Extensão Hash)

Quando o objetivo não é salvar senhas, mas sim gerar tokens, verificar integridade de arquivos ou autenticar trocas de dados entre sistemas, como APIs, usa-se a extensão Hash:

- `hash()`: Cria resumos irreversíveis a partir de um texto.
- `hash_hmac()`: Cria um Código de Autenticação de Mensagem (HMAC) usando uma chave secreta, garantindo que os dados em trânsito não foram alterados.
- `hash_equals()`: Função obrigatória para comparar hashes e tokens. Diferente do sinal normal de igualdade (`==`), ela compara os dados em "tempo constante", impedindo que um invasor descubra a informação monitorando os milissegundos que o processador gasta para fazer a comparação.

```
class Password {
    public static create($password, $options = []) {
        // Esta função retorna um hash exemplo: $2y$07$BCryptRequires22Chrcte/VlQH0piJtjXl.0t1XkA8pw9dMXTP0q
        return password_hash($password, PASSWORD_DEFAULT, $options);
    }

    public function verify($password, $hash) {
        // password_verify verifica se a senha equivale a hash.
        return password_verify($password, $hash);
    }
}

$pwd = "HouseCursos"; // Uma string de senha qualquer
$hash = Password::create($pwd); // Criando um hash para a minha senha
$hash2 = Password::create($pwd, ['cost' => 12]); // Criando um hash com o custo maior.
Password::verify($pwd, $hash); // Verifica se a senha equivale ao hash, caso TRUE senha está correta.
```

FONTES

CAMPITELLI, Vinícius. **Criptografia com PHP**. Disponível em: <https://pt.slideshare.net/slideshow/criptografia-com-php/91084035#1>. Acesso em 2 ago. 2026

CRIPTOGRAFIA em PHP nos formatos: MD5, crypt e base64. 2019. Disponível em: <<https://melhorweb.com.br/criptografia-em-php-nos-formatos-md5-crypt-e-base64/>>. Acesso em 2 ago. 2026.

DONIZETTI, Guilherme. **AES e PHP: Criptografia de dados.** 2022. Disponível em: <<https://guilhermedonizettiads.medium.com/aes-e-php-criptografia-de-dados-292353e38ba2>>. Acesso em 2 ago. 2026.

ZIMUEL, Enrico. **Criptografia segura no PHP.** 2011. Disponível em: <<https://imasters.com.br/back-end/criptografia-segura-no-php>>. Acesso em 2 ago. 2026.